Local AI Lab

Questions for the Report (Part 18)

Local LLM inference with Ollama: application architecture, Retrieval-Augmented Generation, experimental model comparison and security considerations

Author	Siham Yamlahi Alami
Status	PhD Student
Laboratory	LabTIC
Institution	ENSA Tangier
Date	October 25, 2026

Contents of this report

Section 1 — Local AI fundamentals: local vs. cloud LLMs, Ollama, quantization, model size, trade-offs
Section 2 — Application architecture: Flask, REST API and RAG
Section 3 — Multimodal models and experimental comparison of two local models
Section 4 — Security of local LLM servers

Table of Contents

Part 18 — Questions for the Report **3**

Section 1 — Local AI Fundamentals 3

Section 2 — Application Architecture 4

Section 3 — Multimodal Models and Model Comparison 5

Section 4 — Security 6

Part 18 — Questions for the Report

Questions 1 to 15

Section 1 — Local AI Fundamentals

Q1. What is Local AI?

Local AI refers to artificial intelligence models that run directly on a personal computer or local server instead of relying on cloud services. The model uses local hardware (CPU, GPU, and RAM) to process requests and generate responses.

Q2. Difference between a Cloud LLM and a Local LLM?

A Cloud LLM runs on remote servers and requires an internet connection to communicate with the service. A Local LLM runs directly on the user's computer and can work offline once the model and necessary software are installed.

Q3. What role does Ollama play?

Ollama is a tool that simplifies downloading, managing, and running LLMs locally. It provides a REST API that allows Python applications to communicate with local models and generate responses.

Q4. What is model quantization?

Model quantization is a technique that reduces the numerical precision of model weights, for example, from 32-bit to 8-bit or 4-bit. This reduces memory consumption and can improve inference speed, although it may slightly affect accuracy.

Q5. Why does model size affect memory consumption?

Larger models generally contain more parameters, which require more memory to store and process. However, actual memory consumption also depends on quantization, model architecture, context length, and runtime configuration.

Q6. Why can a smaller model sometimes be preferable?

Smaller models require fewer computational resources, use less memory, and can provide faster responses. They are particularly useful for simple tasks and computers with limited hardware.

Q7. Advantages of local inference?

Local inference offers better data privacy, offline availability, greater control over models, and independence from external API services. It also avoids per-request cloud API costs.

Q8. Disadvantages of local inference?

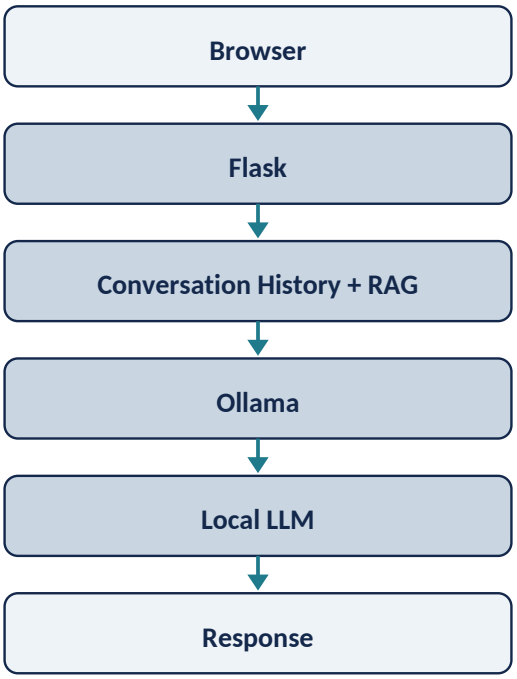
Local inference depends on the computer's hardware capabilities. Limited RAM or GPU memory can reduce performance, and smaller models may produce less accurate answers. Users are also responsible for installation, updates, and maintenance.

Section 2 — Application Architecture

Q9. Explain the architecture of your Python application.

My application uses Flask to provide a web interface where users submit questions. Python manages conversation history and retrieves relevant information from local course documents using a keyword-based retriever. The question and retrieved context are sent to Ollama through its REST API. The local LLM generates an answer, which Flask displays in the browser along with the response time. The application also supports structured JSON responses.

Architecture



Q10. Explain the role of the REST API.

The REST API allows my Python application to communicate with Ollama using HTTP requests. Python sends prompts in JSON format, and Ollama returns the generated responses. This separates the web application from the model inference process.

Q11. What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with text generation. It first searches relevant documents, then provides the retrieved information to the LLM as context. This helps generate answers based on specific sources rather than relying only on the model's training knowledge.

Section 3 — Multimodal Models and Model Comparison

Q12. Difference between an LLM and a multimodal LLM?

A traditional text-based LLM processes and generates text. A multimodal LLM can process multiple types of information, such as text and images. For example, I tested Qwen3.5:0.8b to analyze an image of a cat.

Q13. Compare two local models experimentally.

I compared **Qwen3.5:0.8b** and **Gemma3:1b** using Ollama on my NVIDIA GeForce MX350 GPU with 2 GB VRAM.

Experimental results (Part 16)

Metric	Qwen3.5:0.8b	Gemma3:1b
Disk size (GB)	1.32	0.81
Loaded size (GB)	0.51	0.90
Reported VRAM (GB)	0.51	0.07
Load time (s)	14.17	14.64
Response time (s)	76.96	**58.46**
Generated tokens	706	782
Generation speed (tokens/s)	11.41	**17.98**
System RAM usage (%)	71.3	79.2

Interpretation

Gemma3:1b achieved better performance, generating 17.98 tokens per second compared to 11.41 for Qwen3.5:0.8b. It also completed the task faster (58.46 seconds versus 76.96 seconds).

However, Gemma3 had a larger loaded memory size (0.90 GB versus 0.51 GB). The reported VRAM measurements suggest different CPU/GPU memory allocations during the benchmark.

In the previous programming experiment, Gemma3 also generated more accurate Python code and passed all 2,005 test cases, while Qwen3.5 failed one case.

> **Conclusion**
> Gemma3:1b performed better in speed and code correctness in my experiments, while Qwen3.5:0.8b had a smaller reported loaded memory footprint.

Q14. Which model would you deploy on a computer with limited RAM? Explain.

I would choose **Qwen3.5:0.8b** if minimizing loaded memory consumption is the main priority. In my benchmark, it required approximately 0.51 GB compared to 0.90 GB for Gemma3:1b.

However, if the computer has sufficient memory, I would prefer Gemma3:1b because it provided faster responses and better coding results in my experiments.

Section 4 — Security

Q15. What security risks exist when exposing a local LLM server to a network?

Exposing a local LLM server to a network creates several security risks. Since Ollama does not provide authentication by default, unauthorized users could access the API, consume computing resources, or interact with available models.

Configuring `OLLAMA_HOST=0.0.0.0` allows Ollama to listen on all network interfaces, potentially making it accessible to other devices.

Other risks include exposing sensitive information through application responses and prompt injection attacks through untrusted documents.

To improve security, I would keep Ollama accessible only through `localhost` when possible, configure firewall rules, and use a reverse proxy with authentication and HTTPS/TLS if remote access is necessary. These measures help protect the application and its resources.